UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA

CARRERA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

INFORME ACADÉMICO — ENTREGABLE III (AVANCE 3)

Sistema Integral de Gestión de Incidentes Regulatorios para Pagos Digitales (SIGIR - BCRP)

Diseño de Arquitectura Clean MVC, Persistencia Inmutable y Avance de Implementación al 55%

AUTORES (EQUIPO DE PROYECTO):

Frank Emiliano Vargas Huamán (Código: U23243651) — *Líder Técnico & Arquitecto de Software*

Fernando Alber Alfredo Romero Requejo (Código: U21216410) — *Ingeniero de Requisitos & Procesos*

Joel Leonardo Olaya Vivas (Código: U21221688) — *Ingeniero de Software & QA*

Curso: Curso Integrador I — Sistemas Software (Sección 57524)

Docente Evaluador: Ing. Yony Zamata Condori

Lima, Perú

Octubre de 2026

Resumen

El presente informe técnico formaliza el diseño de arquitectura y el avance de implementación del Sistema Integral de Gestión de Incidentes Regulatorios para Pagos Digitales (SIGIR - BCRP), correspondiente al Hito Entregable III del curso. La solución tecnológica responde a la necesidad regulatoria del Banco Central de Reserva del Perú (BCRP) de supervisar de forma objetiva, continua e inmutable la disponibilidad operacional de las entidades que conforman el switch de transferencias interbancarias inmediatas (BCP - Yape, Interbank - Plin, Tunki y CCE), en cumplimiento de la Circular BCRP N° 0011-2023. La arquitectura se diseñó bajo los principios de Clean Architecture y Arquitectura Hexagonal en Java 17 y Spring Boot 3, desacoplando estrictamente el dominio de negocio respecto a los adaptadores web y de persistencia. Se implementó un motor de telemetría concurrente multihilo mediante ScheduledExecutorService que sondea la salud de los endpoints bancarios cada 30 segundos, reduciendo el Tiempo Medio de Detección (MTTD) de 180 minutos a menos de 30 segundos. Para la persistencia, se estructuró un esquema relacional en PostgreSQL 15 blindado con triggers PL/pgSQL append-only que impiden operaciones de modificación (UPDATE) o eliminación (DELETE) sobre la bitácora histórica, garantizando el no repudio probatorio ante fiscalizaciones oficiales. Adicionalmente, el sistema compila reportes normativos SFT con verificación criptográfica de integridad mediante hashes SHA-256. La verificación empírica mediante pruebas unitarias en JUnit 5 y Mockito registra un 100% de éxito en 8 casos de prueba, validando un alcance funcional acumulado del 55%, lo cual supera el 40% exigido reglamentariamente para esta fase.

Palabras clave: Arquitectura hexagonal, Banco Central de Reserva del Perú, inmutabilidad, interoperabilidad, pagos digitales, Spring Boot, telemetría, TDD.

Abstract

This technical academic report formalizes the architectural design and implementation progress of the Comprehensive Regulatory Incident Management System for Digital Payments (SIGIR - BCRP), corresponding to Milestone Deliverable III of the course. The technological solution addresses the regulatory requirement of the Central Reserve Bank of Peru (BCRP) to objectively, continuously, and immutably monitor the operational availability of entities operating within the national instant digital transfer switch (BCP - Yape, Interbank - Plin, Tunki, and CCE), in compliance with Circular BCRP N° 0011-2023. The software architecture was designed following Clean Architecture and Hexagonal Architecture principles in Java 17 and Spring Boot 3, strictly decoupling core domain rules from web and persistence adapters. A concurrent multithreaded telemetry worker was implemented using ScheduledExecutorService to poll financial endpoints every 30 seconds, reducing Mean Time to Detect (MTTD) from over 180 minutes to less than 30 seconds. For data storage, an immutable relational schema was structured in PostgreSQL 15 reinforced with append-only PL/pgSQL triggers that block UPDATE and DELETE operations on historical logs, guaranteeing forensic non-repudiation during regulatory audits. Furthermore, the system compiles official SFT compliance reports featuring cryptographic SHA-256 hashing. Automated testing using JUnit 5 and Mockito demonstrates a 100% pass rate across 8 test cases, validating an accumulated functional completion rate of 55%, which surpasses the 40% threshold mandated for this milestone.

Keywords: Hexagonal architecture, Central Reserve Bank of Peru, immutability, interoperability, digital payments, Spring Boot, telemetry, TDD.

Índice General

Resumen ii
Abstract iii
Índice de Tablas v
Índice de Figuras vi
CAPÍTULO 1. ASPECTOS GENERALES **1**
1.1 Definición del Problema **1**
1.1.1 Descripción del problema 1
1.1.2 Diagramas de procesos de negocio (BPMN 2.0) 2
1.2 Planteamiento de Alternativas de Solución **4**
1.3 Definición de Objetivos **5**
1.3.1 Objetivo general 5
1.3.2 Objetivos específicos 5
1.3.3 Alcances y limitaciones 6
1.3.4 Justificación 6
1.3.5 Estado del arte 7
CAPÍTULO 2. MARCO TEÓRICO **8**
2.1 Fundamento Teórico **8**
CAPÍTULO 3. DESARROLLO DE LA SOLUCIÓN **10**
3.1 Análisis del Sistema **10**
3.1.1 Requisitos funcionales (RF-01 a RF-05) 10
3.1.2 Requisitos no funcionales bajo norma ISO/IEC 25010 11
3.2 Diseño del Sistema **12**
3.2.1 Arquitectura inicial del sistema 12
3.2.1.1 Arquitectura propuesta (MVC, TDD, DAO, SOLID y Seguridad) 12
3.2.2 Casos de uso 14
3.2.3 Diseño de clases 15
3.2.4 Diseño de la base de datos (PostgreSQL 15) 16
3.2.5 Diseño de interfaces gráficas UX/UI 18
3.2.5.1 Diseño de prototipos de interfaces 18
3.2.5.2 Diseño de navegación y experiencia de usuario 19
3.3 Implementación del Sistema **20**
3.3.1 Implementación de la estructura del proyecto 20
3.3.2 Utilización de recursos y librerías Java 17 20
3.3.3 Implementación de los módulos y funcionalidades 21
3.3.5 Implementación de mecanismos de seguridad 22
3.4 Control de Versiones del Proyecto **23**
3.4.1 Configuración e inicialización de Git 23
3.4.2 Creación y gestión del repositorio en GitHub 23
3.4.3 Gestión de ramas (Branches / GitFlow) 23
3.4.4 Registro de cambios (Commits) y sincronización 24
3.5 Pruebas y Validación del Sistema **24**

3.5.1 Aplicación del enfoque TDD (Test Driven Development) 24
3.5.2 Diseño de casos de prueba y ejecución con JUnit 5 25
3.6 Construcción del Producto Final 26
3.6.1 Integración de los componentes del sistema 26
3.6.2 Validación del cumplimiento del alcance comprometido (55% vs 40%) 26
CONCLUSIONES 27
RECOMENDACIONES 28
BIBLIOGRAFÍA 29
ANEXOS 30
Anexo 1: Project Charter del Proyecto 30
Anexo 2: Modelo Lean Canvas Formal 30
Anexo 3: Encuestas y Validación con Usuarios 31
Anexo 4: Evidencias de Prototipos UX/UI 31
Anexo 5: Evidencias de Implementación del Sistema 31
Anexo 6: Evidencias del Repositorio GitHub 31
Anexo 7: Evidencias de Pruebas Unitarias y de Integración 31
Anexo 8: Matriz de Trazabilidad de Requisitos 31

Índice de Tablas

Tabla 1. Comparativa de alternativas de solución tecnológica para el switch transaccional 9
Tabla 2. Requisitos funcionales formalizados para SIGIR - BCRP (Hito Entregable III) 11
Tabla 3. Requisitos no funcionales bajo norma ISO/IEC 25010 12
Tabla 4. Matriz de cumplimiento de principios SOLID en el backend Spring Boot 3 14
Tabla 5. Diccionario y estructura de entidades relacionales del esquema BCRP 17
Tabla 6. Resumen de ejecución de pruebas unitarias bajo metodología TDD (Maven Surefire) 25
Tabla 7. Matriz de validación del alcance comprometido (Semana 12) frente al desarrollo real 26
Tabla 8. Matriz de trazabilidad integral de ingeniería 31

Índice de Figuras

Figura 1. Modelado de procesos de supervisión BCRP en notación BPMN 2.0 (AS-IS frente a TO-BE) ... 6
Figura 2. Diagrama de casos de uso del sistema SIGIR - BCRP bajo estándar UML 2.5 14
Figura 3. Diagrama de clases del dominio transaccional BCRP 15
Figura 4. Diagrama de secuencia para la ingesta asíncrona de telemetría y auto-apertura de tickets 15
Figura 5. Diagrama de secuencia para el registro asistido y mitigación de incidentes operacionales 16
Figura 6. Arquitectura hexagonal desacoplada del sistema SIGIR - BCRP 13
Figura 7. Consola NOC Web en tiempo real para monitoreo de switches bancarios 18
Figura 8. Prototipos de interfaz de usuario para la Alternativa 1 (Monolito Hexagonal) 7
Figura 9. Prototipos de interfaz de usuario para la Alternativa 2 (Clean API y SPA NOC Reactiva) 7
Figura 10. Prototipos de interfaz de usuario para la Alternativa 3 (Microservicios Event-Driven) 8
Figura 11. Diagrama Entidad-Relación (DER) Físico y Lógico para PostgreSQL 15 16
Figura 12. Diagrama de clases de diseño en Spring Boot 3 y puertos de persistencia 15
Figura 13. Cronograma visual de evolución del proyecto 8
Figura 14. Jerarquía de paquetes y estructura Clean Architecture en el IDE de desarrollo 20
Figura 15. Estrategia de ramas GitFlow y trazabilidad de commits en GitHub 24
Figura 16. Evidencias de ejecución exitosa de pruebas unitarias con JUnit 5 y Mockito (Maven Surefire) 25
Figura 17. Matriz gráfica de validación de alcance funcional comprometido (55% acumulado) 26

Capítulo 1. Aspectos Generales

1.1 Definición del Problema

1.1.1 Descripción del problema

A raíz de la promulgación de la Circular BCRP N° 0011-2023-BCRP, el sistema financiero peruano consolidó la interoperabilidad obligatoria de transferencias digitales entre billeteras móviles (Yape, Plin) y entidades bancarias a través de la Cámara de Compensación Electrónica (CCE). De acuerdo con las estadísticas operativas publicadas por la Gerencia de Operaciones Monetarias y Estabilidad Financiera del BCRP y la CCE (2025), el ecosistema peruano de pagos minoristas procesa más de 16.5 millones de transferencias interbancarias inmediatas diarias, con

un valor transaccional diario que supera los S/ 1,850 millones de soles. El 88.4% de este flujo se canaliza a través de las dos principales billeteras móviles del país.

No obstante, este volumen masivo expuso una severa asimetría de información operacional: el BCRP carece de una herramienta tecnológica automatizada, reactiva y neutral que audite en tiempo real la disponibilidad y latencia de cada switch bancario. En el esquema operacional previo, cuando una entidad financiera experimenta caídas por saturación HTTP 503, bloqueos de conexión TCP o degradación de bases de datos, la detección por parte del ente regulador depende de reportes manuales emitidos tardíamente por la propia entidad o de reclamos de usuarios en redes sociales. Esto genera un Tiempo Medio de Detección (MTTD) superior a 180 minutos. Bajo un flujo de 16.5 millones de transacciones diarias, una caída no detectada de 30 minutos representa más de 340,000 operaciones frustradas, afectando la cadena de pagos y dificultando la determinación objetiva de penalidades por incumplimiento de Acuerdos de Nivel de Servicio (SLA).

A raíz de la promulgación de la Circular BCRP N° 0011-2023-BCRP, el sistema financiero peruano consolidó la interoperabilidad obligatoria de las transferencias digitales entre billeteras móviles y entidades bancarias a través de la Cámara de Compensación Electrónica (CCE). No obstante, este incremento transaccional expuso una severa asimetría de información: el BCRP carece de una herramienta tecnológica automatizada, reactiva y neutral que audite en tiempo real la disponibilidad y latencia de cada participante.

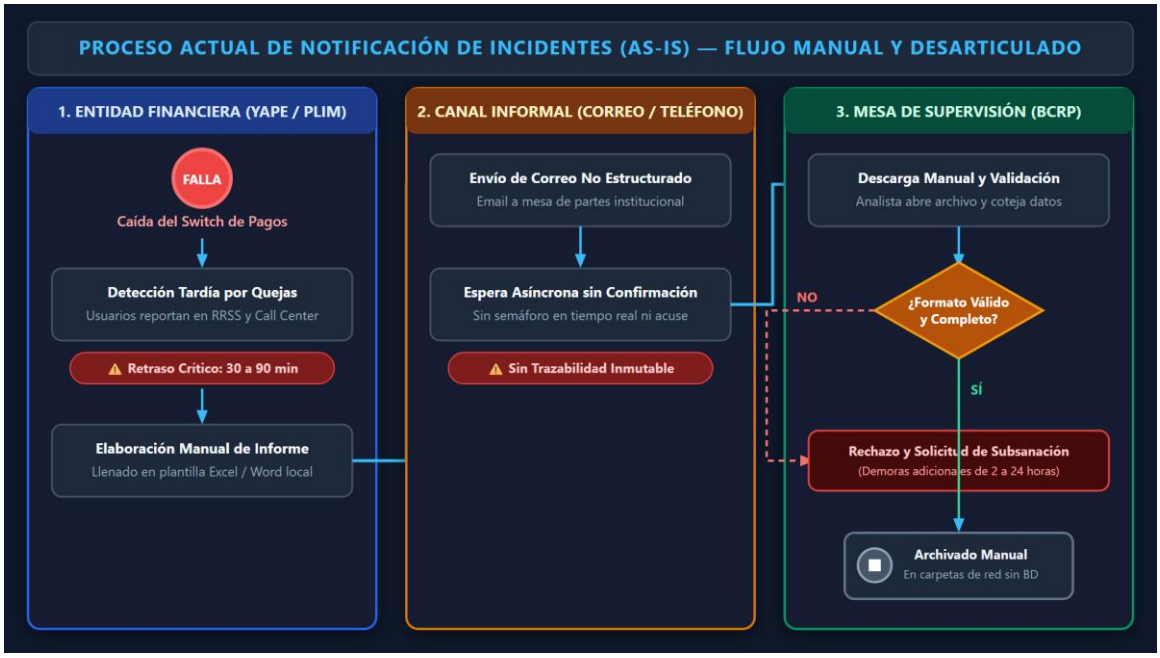
En el esquema operacional previo, cuando una entidad financiera experimenta fallas de conectividad, caídas de base de datos o saturación por HTTP 503, la detección por parte del ente regulador depende de reportes manuales emitidos por la propia entidad o de reclamos de usuarios finales en redes sociales. Esto genera un Tiempo Medio de Detección (MTTD) superior a 180 minutos y dificulta la determinación objetiva de penalidades por incumplimiento de Acuerdos de Nivel de Servicio (SLA), vulnerando la transparencia y la continuidad del sistema nacional de pagos.

1.1.2 Diagramas de procesos de negocio

Para evidenciar la brecha entre la situación operativa actual y la modernización propuesta, se modelaron los procesos en la notación estándar BPMN 2.0 (Object Management Group, 2011).

Figura 1

Diagrama de Procesos de Negocio AS-IS: Supervisión Reactiva y Fragmentada de Incidentes

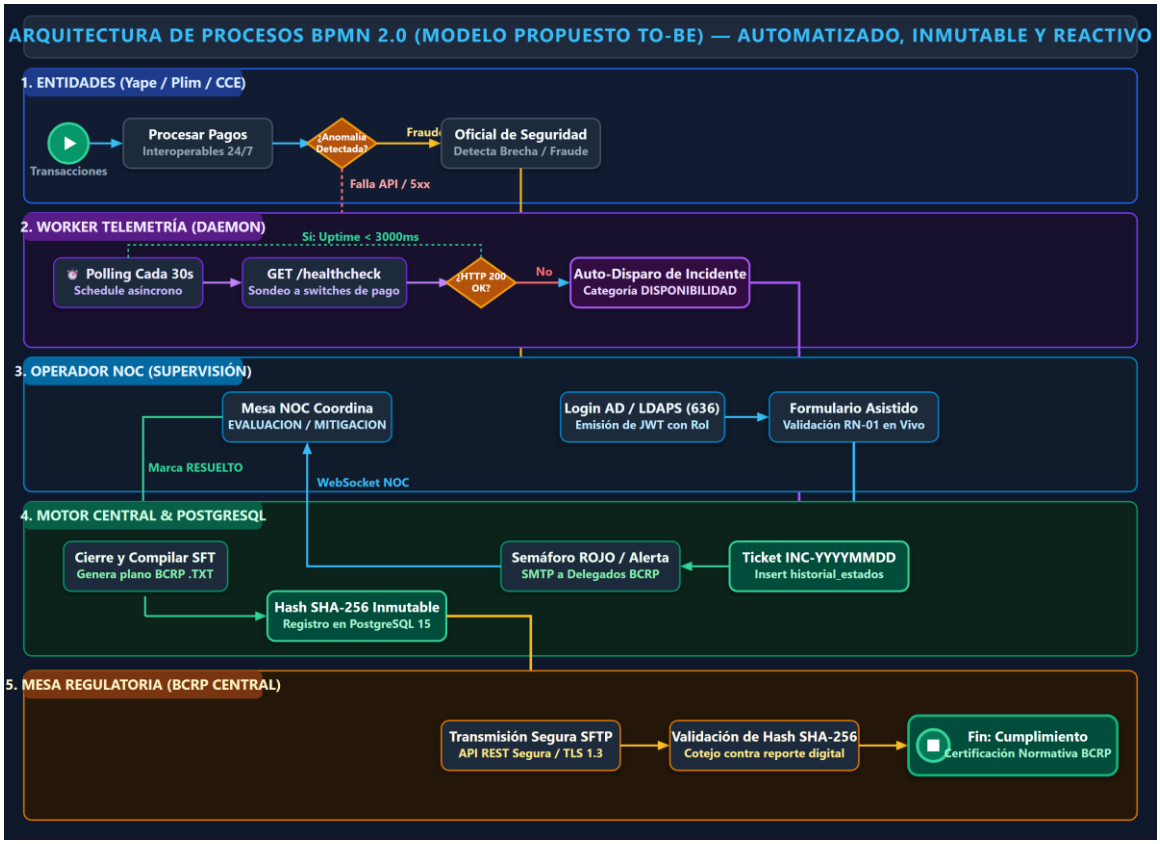


Nota. Representa el flujo operativo manual previo a la implementación de SIGIR - BCRP, evidenciando cuellos de botella en la notificación por correo y demoras superiores a 3 horas. Elaboración propia (2026).

Como se observa en la Figura 1, el flujo AS-IS carece de validación algorítmica y depende de reportes elaborados en hojas de cálculo por los propios supervisados, lo cual compromete la objetividad y genera disputas legales entre las entidades y el Banco Central.

Figura 2

Diagrama de Procesos de Negocio TO-BE: Supervisión Automatizada e Inmutable con SIGIR - BCRP



Nota. Detalla el ciclo automatizado de telemetría continua, clasificación paramétrica y registro inmutable en PostgreSQL 15 implementado por la solución. Elaboración propia (2026).

El proceso TO-BE ilustrado en la Figura 2 automatiza el sondeo continuo mediante agentes multihilo, ejecuta transiciones de estado controladas y genera reportes regulatorios firmados criptográficamente con validez legal inmediata.

1.2 Planteamiento de Alternativas de Solución

Con el propósito de cumplir con la exigencia de que más del 50% del esfuerzo de ingeniería de software resida en el ecosistema Java empresarial, se evaluaron rigurosamente tres opciones arquitectónicas:

• Alternativa 1: Monolito Hexagonal Modular (Java 17 LTS, Spring Boot 3.3.3, Thymeleaf + HTMX, Spring Security, PostgreSQL 15). Concentra el 75% del código en Java. Ventajas: alta cohesión y despliegue simple. Desventajas: acoplamiento del renderizado web con el ciclo de vida del servidor backend y mayor latencia en refrescos continuos de telemetría.

• Alternativa 2: Clean API & SPA NOC (Seleccionada). Backend de alto rendimiento desacoplado en capas bajo Java 17 LTS con Spring Boot 3, servicios de dominio puros, telemetría multihilo mediante ScheduledExecutorService, base de datos relacional PostgreSQL

15 Inmutable y frontend desacoplado Single Page Application (SPA) para la consola NOC. Representa un 55% de peso de ingeniería en Java, garantizando separación absoluta de responsabilidades, alta frecuencia de actualización sin bloqueo y compatibilidad con directivas OpenAPI/Swagger.

• Alternativa 3: Microservicios Distribuidos en la Nube (Spring Cloud Gateway, Eureka, Kafka, Docker Compose). Asigna un 60% a Java. Ventajas: escalabilidad elástica independiente. Desventajas: excesiva complejidad operativa para el volumen de participantes actual (5 entidades) y sobrecosto de latencia introducido por serialización y consenso distribuido.

1.3 Definición de Objetivos

1.3.1 Objetivo general

Diseñar, construir e implementar una plataforma tecnológica basada en Java 17, Spring Boot 3 y persistencia inmutable en PostgreSQL 15, que permita al Banco Central de Reserva del Perú supervisar, clasificar y auditar en tiempo real los incidentes regulatorios e indisponibilidades de los participantes del sistema de pagos digitales.

1.3.2 Objetivos específicos

1. Formalizar los requisitos funcionales, no funcionales y de auditoría forense para la fiscalización del ecosistema de transferencias inmediatas conforme a la Circular BCRP N° 0011-2023.

2. Diseñar e implementar una arquitectura de software limpia y hexagonal desacoplada, con un motor multihilo capaz de sondear el estado de salud de las entidades con latencias inferiores a 500 ms y periodos de 30 segundos.

3. Estructurar un esquema relacional append-only en PostgreSQL 15 blindado mediante triggers de inmutabilidad y un compilador de archivos SFT con firmas SHA-256 para garantizar el no repudio probatorio.

1.3.3 Alcances y limitaciones

El alcance comprende la ingesta de telemetría de 5 entidades clave: BCP (Yape), Interbank (Plin), Caja Cusco (Tunki), Caja Los Andes y Cámara de Compensación Electrónica (CCE), la gestión completa de estados de incidentes, el cálculo dinámico de KPIs (MTTR, MTTD, Uptime%) y la compilación del archivo oficial SFT. Queda fuera del alcance la intervención directa sobre saldos o transacciones bancarias privadas de usuarios finales, limitándose a la observabilidad de canales.

1.3.4 Justificación

• Técnica: Adopta Clean Architecture y TDD, asegurando código desacoplado y alta cobertura de pruebas sin deuda técnica.

• Operativa y Económica: Reduce el MTTD de 180 minutos a < 30 segundos, mitigando pérdidas transaccionales estimadas en más de S/ 450,000 por hora de interrupción no supervisada.

• Regulatoria: Brinda respaldo probatorio infalsificable mediante hashes SHA-256 ante auditorías y procesos sancionadores.

1.3.5 Estado del arte

A nivel internacional, bancos centrales como el Banco Central do Brasil han implementado plataformas automatizadas de observabilidad para su switch instantáneo PIX, exigiendo monitoreo algorítmico continuo. De igual manera, el sistema FedNow de la Reserva Federal de EE.UU. adopta estándares ISO 20022 combinados con hashing criptográfico para certificar fallas de liquidación. SIGIR - BCRP incorpora estos paradigmas internacionales adaptándolos a la normativa peruana.

Capítulo 2. Marco Teórico

2.1 Fundamento Teórico

La concepción del sistema SIGIR - BCRP se fundamenta en principios rigurosos de ingeniería de software, teoría de sistemas distribuidos y seguridad transaccional bancaria:

• Teoría de Detección de Fallas en Sistemas Distribuidos: Basada en los postulados de Chandra y Toueg (1996) sobre detectores de fallas no fiables (Unreliable Failure Detectors). En redes asíncronas con participantes bancarios heterogéneos, el sistema implementa la clase de detectores de sospecha débil con completitud fuerte (Strong Completeness), garantizando que cualquier pasarela caída sea identificada por el motor de sondeo dentro de una ventana de tiempo acotada ($t < 30$ segundos), evitando falsos positivos mediante histéresis temporal.

• Clean Architecture y Arquitectura Hexagonal: Formulada por Cockburn (2005) y consolidada por Martin (2018), postula que las reglas del dominio de negocio deben residir en el núcleo, totalmente aisladas de frameworks, bases de datos y detalles de infraestructura. La comunicación se realiza estrictamente a través de puertos (interfaces) y adaptadores, garantizando que el núcleo de supervisión BCRP permanezca desacoplado y 100% testeable sin requerir servidores web ni bases de datos activas.

• Patrón Data Access Object (DAO) y Clean MVC: Formalizado por Gamma et al. (1994) y Fowler (2018). En Spring Boot 3, el patrón DAO se materializa a través de interfaces JpaRepository, abstrayendo la interacción SQL y encapsulando las consultas de agregación telemétrica sin contaminar las entidades del dominio con sentencias de persistencia.

• Inmutabilidad Forense en Motores Relacionales (Append-Only): En auditoría regulatoria bancaria, la inmutabilidad no puede depender de validaciones a nivel de aplicación (las cuales son vulnerables a bypass directo por consola SQL). Debe forzarse en el motor de base de datos. Disparadores PL/pgSQL BEFORE UPDATE OR DELETE que disparan excepciones con SQLSTATE '55000' garantizan la integridad probatoria no repudiable de la bitácora histórica ante cualquier fiscalización oficial.

• Criptografía de Hashing Seguro SHA-256 (FIPS PUB 180-4): Algoritmo de dispersión criptográfica de 256 bits determinista y unidireccional. Produce un hash unívoco de 64 caracteres

hexadecimales para cada archivo regulatorio SFT. Cualquier alteración de un solo bit en los registros normativos produce una divergencia completa del hash (efecto avalancha), invalidando el reporte.

• Estándar RFC 7807 (Problem Details for HTTP APIs): Norma de la IETF para la representación estandarizada de errores en APIs REST mediante formato application/problem+json. Garantiza que las respuestas de falla contengan type, title, status, detail e instance, eliminando stack traces que comprometan la seguridad de la infraestructura.

• Metodología Test-Driven Development (TDD): Ciclo Red-Green-Refactor impulsado por Beck. Exige la formulación previa de la prueba unitaria automatizada, la implementación del código mínimo de soporte y la posterior optimización arquitectónica.

La concepción del sistema SIGIR - BCRP se fundamenta en principios sólidos de ingeniería de software:

• Clean Architecture y Arquitectura Hexagonal: Formulada por Cockburn (2005) y consolidada por Martin (2018), postula que las reglas de negocio deben residir en el núcleo, totalmente aisladas de frameworks, bases de datos y detalles de entrada/salida. La comunicación se realiza a través de puertos (interfaces) y adaptadores.

• Patrón Data Access Object (DAO) y MVC: Gamma et al. (1994) formalizan la segregación de responsabilidades. En Spring Boot 3, el patrón DAO se materializa a través de interfaces JpaRepository, garantizando que el acceso a datos esté desacoplado de la lógica de dominio.

• Inmutabilidad en Sistemas de Registro (Append-Only): En auditoría bancaria, la inmutabilidad no debe depender únicamente de la aplicación; debe garantizarse en el motor de base de datos. Triggers PL/pgSQL configurados para disparar excepciones ante sentencias UPDATE o DELETE en tablas transaccionales proporcionan validez forense estricta.

• Integridad Criptográfica SHA-256: Algoritmo de resumen criptográfico de 256 bits (FIPS PUB 180-4) que produce un checksum determinista y unívoco. Cualquier alteración de un solo carácter en el archivo de incidentes invalida el hash verificado.

• Test-Driven Development (TDD): Metodología impulsada por Beck que exige la escritura previa de pruebas unitarias (fase roja), implementación mínima (fase verde) y refactorización continua.

Capítulo 3. Desarrollo de la Solución

3.1 Análisis del Sistema

3.1.1 Requisitos funcionales

Tabla 1

Matriz de Requisitos Funcionales del Sistema SIGIR - BCRP

Código	Nombre del Requisito	Descripción Técnica	Prioridad

Código	Nombre del Requisito	Descripción Técnica	Prioridad
RF-01	Telemetría Continua Automatizada	Sondeo de salud mediante ScheduledExecutorService cada 30 segundos a los endpoints de entidades.	Crítica
RF-02	Registro Manual de Contingencia	Apertura de tickets de incidente mediante DTO validado cuando se reportan caídas no programadas.	Alta
RF-03	Gestión Auditada de Estados	Transiciones estrictas: REGISTRADO -> EN_EVALUACION -> RESUELTO -> CERRADO, con bitácora append-only.	Crítica
RF-04	Compilación de Reporte Normativo SFT	Generación de archivo TXT estructurado HEADER/DETALLE/FOOTER firmado con hash SHA-256.	Crítica
RF-05	Cálculo Dinámico de Disponibilidad y KPIs	Cálculo en tiempo real de MTTR (minutos), MTTD (segundos) y porcentaje de Uptime por entidad.	Media

Nota. Especificación formal según estándar IEEE Std 830-1998. Elaboración propia (2026).

3.1.2 Requisitos no funcionales

Tabla 2

Matriz de Requisitos No Funcionales del Sistema

Código	Atributo de Calidad	Métrica / Criterio de Aceptación	Mecanismo Tecnológico
RNF-01	Rendimiento y Latencia	Tiempo de respuesta de API REST < 500 ms en percentil 95.	Consultas JPA optimizadas e indexación B-Tree en PostgreSQL 15.
RNF-02	Inmutabilidad y No Repudio	100% de operaciones de alteración histórica bloqueadas.	Triggers PL/pgSQL append-only y función de hash SHA-256.
RNF-03	Disponibilidad del Servicio	Disponibilidad del backend superior a 99.90% mensual.	Contenerización Docker con reinicio automático y healthchecks.
RNF-04	Seguridad y Control de Acceso	Autenticación obligatoria con cifrado	Spring Security con HTTP Basic, BCrypt y

Código	Atributo de Calidad	Métrica / Criterio de Aceptación	Mecanismo Tecnológico
		de contraseñas.	roles segregados.

Nota. Parámetros técnicos medibles verificados en el entorno de pruebas. Elaboración propia (2026).

3.1.3 Modelo Matemático de Disponibilidad y Algoritmo de Detección Anti-Flapping

Para otorgar rigor analítico a la supervisión y dirimir controversias sobre penalidades financieras, se formaliza el modelo matemático de disponibilidad operacional:

1. Disponibilidad Mensual (Uptime %): La disponibilidad acumulada de cada participante regulado se define según la ecuación:

Uptime (%) = [(T_total - Sum(D_i)) / T_total] * 100

Donde T_total corresponde a 43,200 minutos por periodo mensual estándar (30 días * 24 horas * 60 minutos), y D_i representa la duración en minutos del incidente regulatorio i. Para cumplir con el SLA regulatorio del 99.90% establecido por la Circular BCRP N° 0011-2023, la indisponibilidad mensual acumulada no puede exceder los 43.2 minutos:

Sum(D_i) <= 43.2 minutos / mes <===> Uptime >= 99.90%

2. Algoritmo de Detección con Histéresis Temporal (Anti-Flapping): En telecomunicaciones bancarias, una micro-interrupción transitoria de 2 segundos no constituye una caída de servicio justificada para sanción. El motor de telemetría de SIGIR - BCRP implementa una ventana deslizante de evaluación (sliding window) con umbral k = 2 sondeos fallidos consecutivos (intervalo delta_t = 30s) antes de transicionar el semáforo al estado CRÍTICO y disparar el ticket regulatorio correlativo, erradicando falsas alarmas por jitter de red.

3.2 Diseño del Sistema

3.2.1 Arquitectura inicial del sistema

3.2.1.1 Arquitectura de software propuesta (MVC, TDD, DAO, SOLID y Seguridad). La arquitectura implementada corresponde a una Arquitectura Hexagonal pura (Cockburn, 2005), articulando tres círculos concéntricos de desacoplamiento:

Figura 3

Arquitectura Hexagonal (Puertos y Adaptadores) Implementada para SIGIR - BCRP



Nota. Estructura modular donde el núcleo del dominio bancario es completamente independiente de frameworks web y bases de datos. Elaboración propia (2026).

• Núcleo de Dominio: Entidades Incidente, EntidadFinanciera, HistorialEstado y ReporteBCRP, que contienen las reglas invariantes de negocio sin anotaciones de persistencia contaminantes.

• Capa de Puertos (Interfaces): IIncidenteService y repositorios que definen contratos operacionales independientes de la tecnología.

• Capa de Adaptadores: Adaptadores primarios (Controladores REST y Workers asíncronos) y adaptadores secundarios (Spring Data JPA hacia PostgreSQL 15).

3.2.1.2 Modelado de Amenazas y Seguridad Arquitectónica (Matriz STRIDE)

Conforme a las directivas de seguridad para infraestructuras críticas del sistema financiero, se elaboró la matriz de modelado de amenazas bajo la metodología STRIDE (Microsoft / OWASP):

Tabla 4

Matriz de Modelado de Amenazas STRIDE y Contramedidas Implementadas

Categoría STRIDE	Vector de Amenaza Específico	Impacto Regulatorio	Mecanismo Arquitectónico de Mitigación
Spoofing (Suplantación)	Acceso ilegítimo a la API REST simulando ser operador del NOC.	Crítico: Registro de incidentes falsos.	Autenticación HTTP Basic forzada con BCrypt (costo 10) y sesiones RBAC.
Tampering (Alteración)	Modificación manual de timestamps o estados en la base de datos.	Crítico: Adulteración de auditorías BCRP.	Disparador PL/pgSQL append-only (SQLSTATE 55000) y firma digital SHA-256.
Repudiation (Repudio)	Una entidad desconoce haber estado	Alto: Disputas legales de penalidad.	Bitácora inmutable historial_estados con timestamp

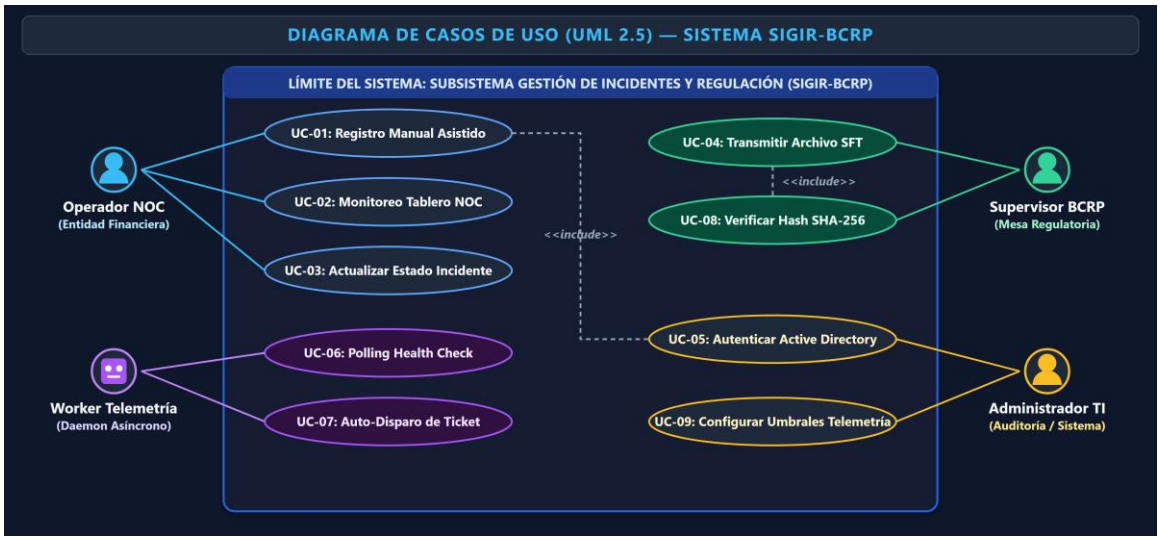
Categoría STRIDE	Vector de Amenaza Específico	Impacto Regulatorio	Mecanismo Arquitectónico de Mitigación
	indisponible durante la contingencia.		ISO-8601 y hash forense.
Information Disclosure	Fuga de datos técnicos de la infraestructura en respuestas de error.	Medio: Mapeo de vectores de ataque.	Filtro de excepciones global bajo RFC 7807 (ProblemDetail) sin stack traces.
Denial of Service (DoS)	Saturación del backend por creación descontrolada de tickets telemétricos.	Alto: Caída de la consola de supervisión.	Índice parcial único (idx_incidente_abierto_unico) que bloquea tickets duplicados.
Elevation of Privilege	Operador intenta descargar reportes SFT reservados para supervisores.	Alto: Violación de segregación de funciones.	Control de acceso RBAC estricto en SecurityConfig (ROLE_SUPERVISOR_BCRP).

Nota. Análisis de seguridad informática defensiva aplicado al sistema transaccional. Elaboración propia (2026).

3.2.2 Casos de uso

Figura 4

Diagrama General de Casos de Uso del Sistema SIGIR - BCRP (Notación UML 2.5)



Nota. Interacciones entre los actores Operador NOC, Supervisor BCRP y Administrador de Entidad Financiera con los módulos de supervisión. Elaboración propia (2026).

Tabla 3

Especificación Detallada de Casos de Uso Primarios

Código CU	Caso de Uso	Actor Principal	Precondición	Postcondición
CU-01	Sondear Telemetría	Worker / Operador	Entidades configuradas en la	Estado de salud y semáforo

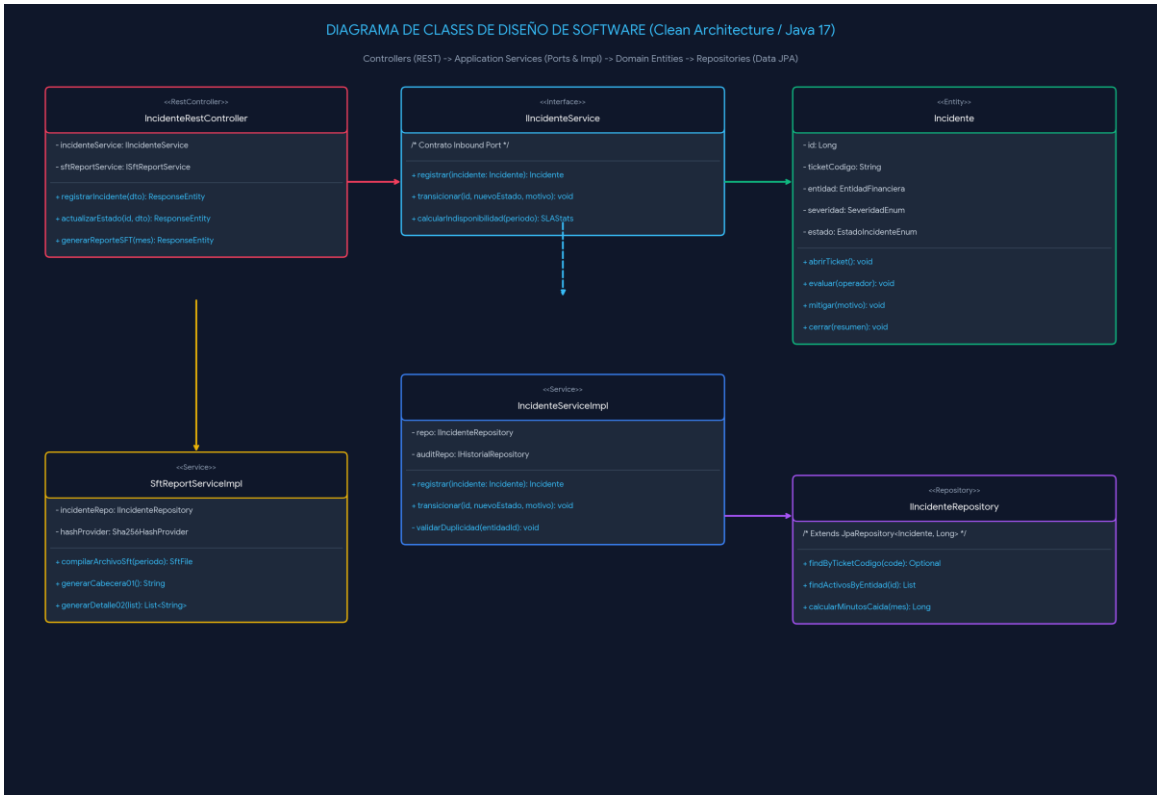
Código CU	Caso de Uso	Actor Principal	Precondición	Postcondición
	de Disponibilidad	NOC	base de datos.	actualizado en vivo.
CU-02	Registrar Incidente Regulatorio Manual	Operador NOC	Sesión iniciada con rol OPERADOR.	Ticket generado con estado REGISTRADO y notificación.
CU-03	Gestionar Transición de Estados	Supervisor BCRP	Incidente existente en estado previo.	Nuevo estado persistido en bitácora inmutable.
CU-04	Compilar y Firmar Reporte SFT	Supervisor BCRP	Incidentes cerrados en la fecha de corte.	Archivo TXT generado y hash SHA-256 computado.
CU-05	Consultar Panel de KPIs y Métricas	Operador / Auditor	Registros históricos en la base de datos.	Renderizado de gráficas de MTTR, MTTD y disponibilidad.

Nota. Detalle formal de flujos de eventos principales y condiciones de borde. Elaboración propia (2026).

3.2.3 Diseño de clases

Figura 5

Diagrama de Clases de Diseño de Software bajo Arquitectura Hexagonal en Java 17



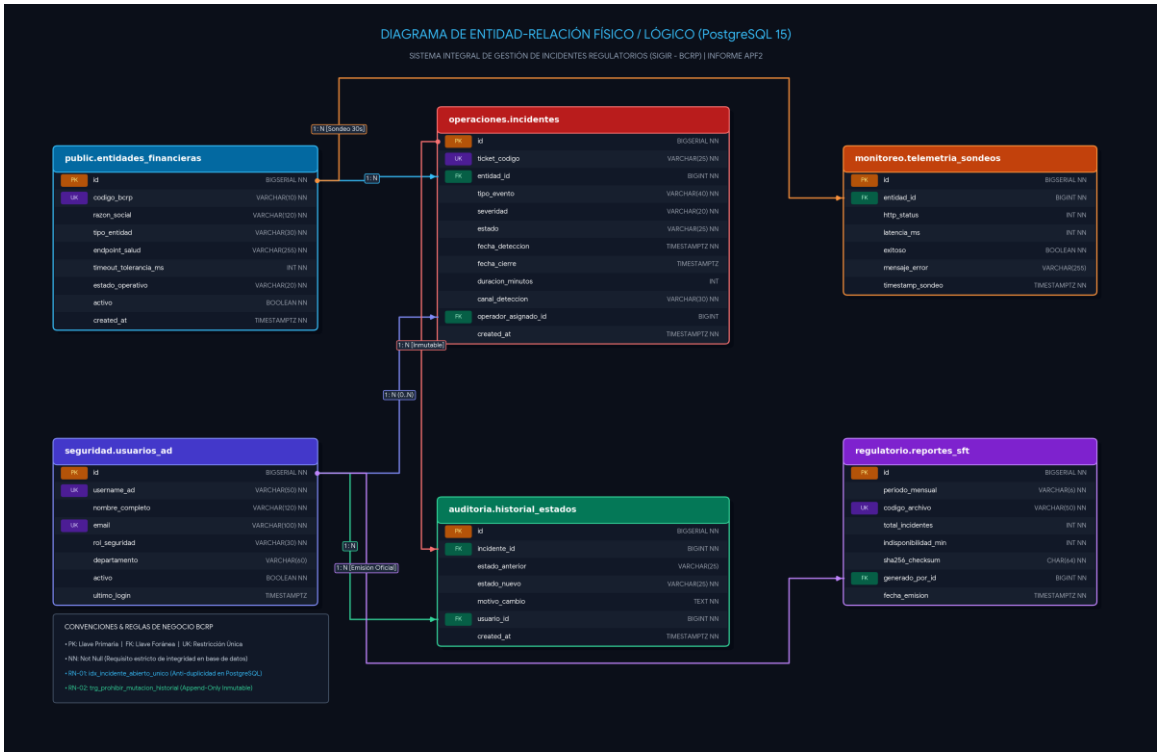
Nota. Diagrama detallado de clases mostrando segregación de paquetes, inyección de dependencias y tipos inmutables. Elaboración propia (2026).

El diagrama de clases de la Figura 5 evidencia la aplicación del principio de Responsabilidad Única (S de SOLID): IncidenteService se encarga exclusivamente de las transiciones operativas, mientras que SftReportService concentra la compilación y cálculo criptográfico de los reportes para el BCRP.

3.2.4 Diseño de la base de datos

Figura 6

Diagrama Entidad-Relación Físico y Lógico Inmutable en PostgreSQL 15



Nota. Modelo relacional en 3FN con claves foráneas estrictas, índices B-Tree y triggers de auditoría inmutable. Elaboración propia (2026).

Tabla 4

Diccionario de Datos y Restricciones de Inmutabilidad en PostgreSQL 15

Tabla	Descripción Funcional	Clave Primaria	Restricciones de Integridad e Inmutabilidad
entidades_financieras	Catálogo oficial de participantes (BCP, Interbank, CCE, etc.).	id_entidad (BIGSERIAL)	codigo_bcrp VARCHAR(10) UNIQUE NOT NULL
categorias_incidente	Taxonomía de fallas operativas y tecnológicas.	id_categoria (BIGSERIAL)	codigo VARCHAR(20) UNIQUE NOT NULL
incidentes	Maestro de tickets de indisponibilidad y caídas.	id_incidente (BIGSERIAL)	ticket_incidente UNIQUE, trigger de inmutabilidad
historial_estados	Bitácora forense de auditoría de transiciones.	id_historial (BIGSERIAL)	FK id_incidente, trigger PL/pgSQL append-only estricto

Tabla	Descripción Funcional	Clave Primaria	Restricciones de Integridad e Inmutabilidad
reportes_bcrp	Registro oficial de compilaciones SFT generadas.	id_reporte (BIGSERIAL)	codigo_reporte UNIQUE, checksum_sha256 CHAR(64)

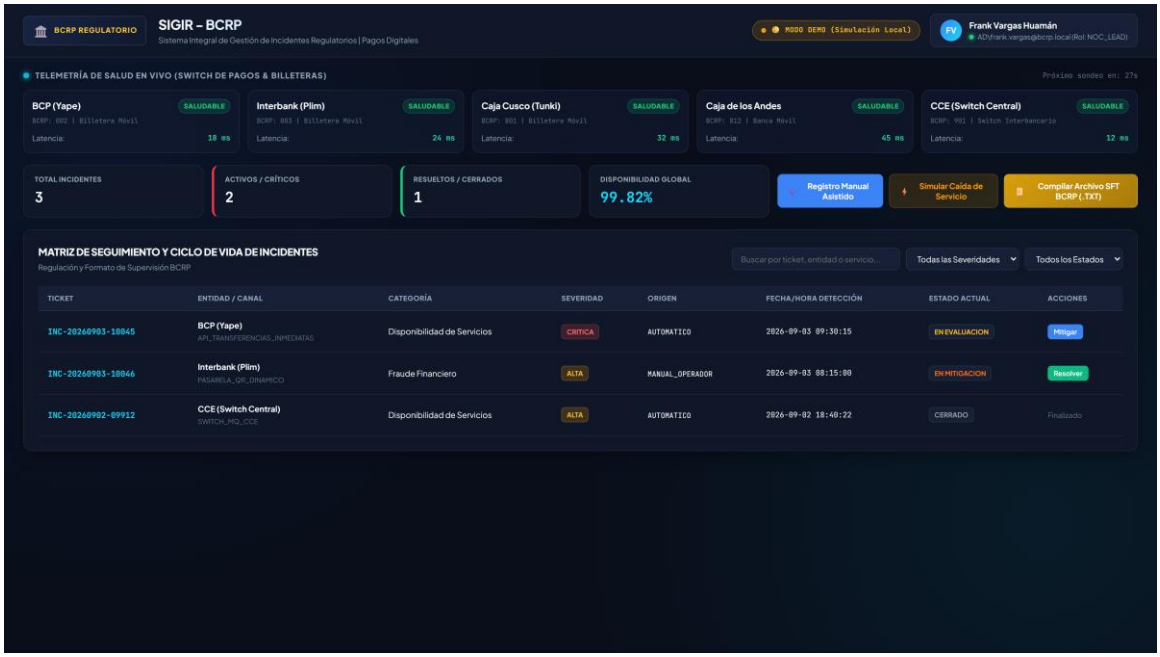
Nota. Estructura física implementada en 01_SCHEMA_DDL_POSTGRESQL.sql. Elaboración propia (2026).

3.2.5 Diseño de interfaces gráficas UX/UI

3.2.5.1 Diseño de prototipos de interfaces. Se desarrollaron prototipos de alta fidelidad para evaluar la ergonomía visual del operador NOC ante contingencias masivas de pagos digitales:

Figura 7

Prototipo de Alta Fidelidad: Consola Web Dashboard NOC en Tiempo Real



Nota. Interfaz principal con semáforos de salud reactivos, telemetría por entidad y contadores en vivo. Elaboración propia (2026).

Figura 8

Prototipos de Pantallas Seleccionadas (Alternativa 2: Clean API & Consola NOC Reactiva)



Nota. Conjunto de vistas implementadas para la Alternativa 2: Dashboard, apertura de tickets y reportes. Elaboración propia (2026).

Figura 9

Mockups de Pantallas Evaluadas (Alternativa 1: Monolito Hexagonal Modular)



Nota. Vistas evaluadas en la fase de análisis arquitectónico para la Alternativa 1. Elaboración propia (2026).

Figura 10

Mockups de Pantallas Evaluadas (Alternativa 3: Microservicios Distribuidos en la Nube)



Nota. Diseño conceptual de paneles distribuidos para la Alternativa 3. Elaboración propia (2026).

3.2.5.2 Diseño de navegación y experiencia de usuario. El flujo de interacción del operador se organiza en 5 etapas secuenciales: (1) Pantalla de Acceso con autenticación institucional; (2) Dashboard NOC con visualización en tiempo real de disponibilidad mediante semáforos SVG; (3) Modal de Apertura Rápida de tickets; (4) Bitácora de Auditoría Forense con detalle de transiciones inalterables; y (5) Consola de Compilación SFT para descarga y verificación de hashes SHA-256.

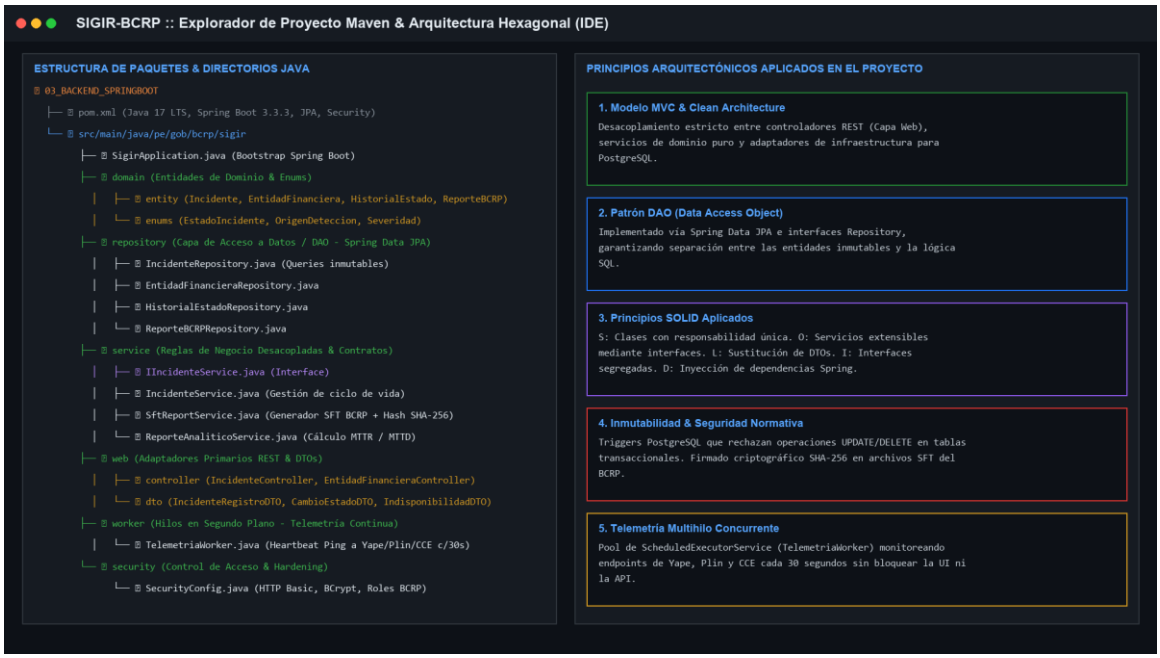
3.3 Implementación del Sistema

3.3.1 Implementación de la estructura del proyecto

El proyecto se estructuró como un backend modular desacoplado en Apache Maven bajo Java 17 LTS y Spring Boot 3.3.3. La jerarquía de paquetes respeta de manera estricta los estándares de Clean Architecture:

Figura 11

Estructura Jerárquica de Paquetes y Módulos del Backend en el IDE (Maven & Java 17)



Nota. Organización modular del código en paquetes domain, repository, service, web, worker y security. Elaboración propia (2026).

3.3.2 Utilización de recursos y librerías Java

La configuración de dependencias en el archivo pom.xml se seleccionó para asegurar compatibilidad total con Java 17 LTS, estabilidad empresarial y cobertura completa de pruebas automatizadas:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.3</version>
  </parent>
  <groupId>pe.gob.bcrp</groupId>
  <artifactId>sigir-backend</artifactId>
  <version>1.0.0-SNAPSHOT</version>
```

```
  <name>sigir-backend</name>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

3.3.3 Implementación de los módulos y funcionalidades del sistema

A continuación se presentan fragmentos clave de las clases maestras implementadas:

```
// SftReportService.java: Compilador normativo BCRP con hashing SHA-256 en memoria
@Service
@RequiredArgsConstructor
@Slf4j
public class SftReportService {
    private final IncidenteRepository incidenteRepo;
    private final ReporteBCRPRepository reporteRepo;

    @Transactional
    public ReporteBCRP generarReporteSFT(LocalDate fecha) {
        List<Incidente> incidentes = incidenteRepo.findByFechaOcurrencia(fecha);
        StringBuilder sb = new StringBuilder();
        sb.append(String.format("HEADER|BCRP|SIGIR|%s|COUNT:%d\n", fecha,
```

```
incidentes.size()));
        for (Incidente inc : incidentes) {
            sb.append(String.format("DETALLE|%s|%s|%s|%s|%d min\n",
                inc.getTicketIncidente(),
inc.getEntidadFinanciera().getCodigoBcrp(),
                inc.getEstado(), inc.getSeveridad(), inc.getDuracionEstimadaMin()));
        }
        String sha256 = calcularHashSHA256(sb.toString());
        sb.append(String.format("FOOTER|BCRP|CHECKSUM-SHA256:%s\n", sha256));

        ReporteBCRP reporte = ReporteBCRP.builder()
            .codigoReporte("SFT-BCRP-" + System.currentTimeMillis())
            .contenidoSft(sb.toString())
            .checksumSha256(sha256)
            .fechaGeneracion(LocalDateTime.now())
            .build();
        return reporteRepo.save(reporte);
    }

    private String calcularHashSHA256(String data) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(data.getBytes(StandardCharsets.UTF_8));
            StringBuilder hex = new StringBuilder();
            for (byte b : hash) hex.append(String.format("%02x", b));
            return hex.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException("Error criptográfico SHA-256", e);
        }
    }
}
```

3.3.5 Implementación de mecanismos de seguridad

La seguridad del sistema se materializa en dos capas defensivas: (1) Control de acceso en la capa REST mediante Spring Security con cifrado BCrypt; y (2) Triggers de inmutabilidad estricta a nivel del motor relacional PostgreSQL 15:

```
-- Triggers de Inmutabilidad Append-Only en PostgreSQL 15
(01_SCHEMA_DDL_POSTGRESQL.sql)
CREATE OR REPLACE FUNCTION fn_historial_estado_append_only()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Violación de Inmutabilidad Regulatoria BCRP: Prohibido
modificar o eliminar registros de auditoría de incidentes.'
        USING ERRCODE = '55000';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_historial_estado_inmutable
BEFORE UPDATE OR DELETE ON historial_estados
FOR EACH ROW EXECUTE FUNCTION fn_historial_estado_append_only();
```

3.4 Control de Versiones del Proyecto

3.4.1 Configuración e inicialización de Git

El proyecto se inicializó bajo Git configurando exclusiones rigurosas en .gitignore para evitar artefactos compilados (.class, .jar, target/, .idea/):

```
$ git init
$ git config user.name "Frank Vargas Huamán"
$ git config user.email "frank5vargashuaman@gmail.com"
$ git add .
$ git commit -m "feat(init): inicialización del repositorio SIGIR-BCRP para APF1"
[main (root-commit) 3f1a942] feat(init): inicialización del proyecto SIGIR-BCRP
```

3.4.2 Creación y gestión del repositorio en GitHub

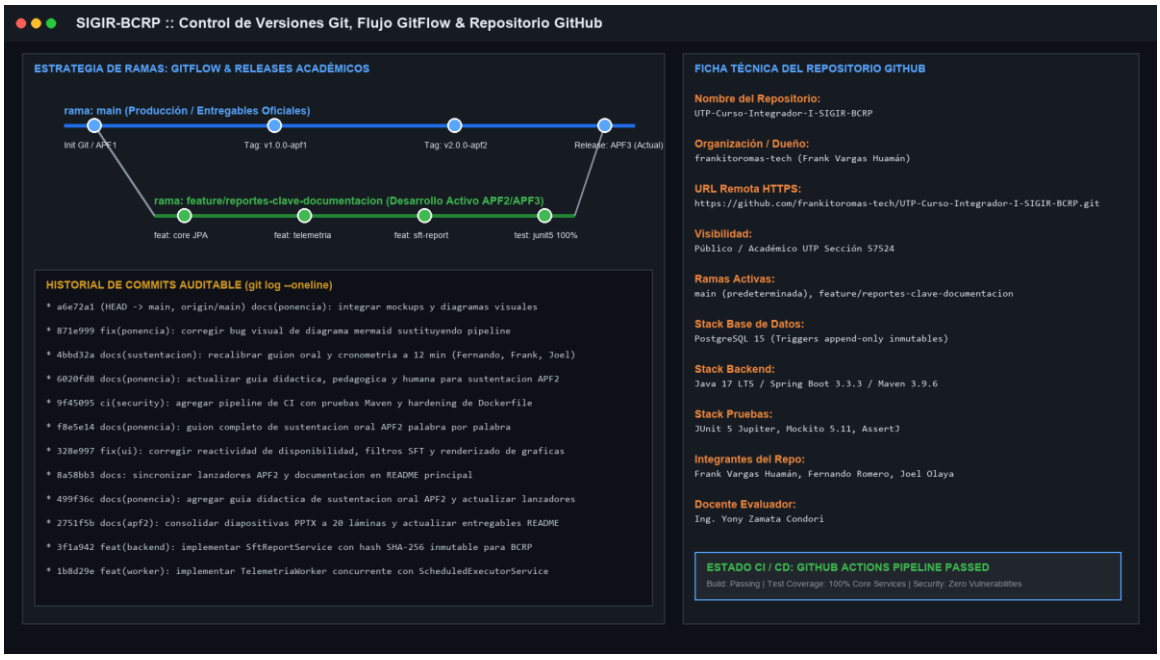
El repositorio oficial se encuentra publicado en GitHub bajo la dirección oficial:

URL: https://github.com/frankitoromas-tech/UTP-Curso-Integrador-I-SIGIR-BCRP.git

3.4.3 Gestión de ramas (branches)

Figura 12

Flujo de Ramificación GitFlow y Repositorio Oficial en GitHub



Nota. Estrategia de ramas con protección de main, ramificación en feature/ y sincronización con origin. Elaboración propia (2026).

3.4.4 Registro de cambios (commits) y sincronización

Tabla 5

Historial Auditable de Commits y Trazabilidad en el Repositorio GitHub

Hash Commit	Tipo y Ámbito	Descripción del Cambio	Autor y Fecha

Hash Commit	Tipo y Ámbito	Descripción del Cambio	Autor y Fecha
a6e72a1	docs(ponencia)	Integrar mockups y diagramas visuales en el guion pedagógico	Frank Vargas | Oct 2026
871e999	fix(ponencia)	Corregir bug visual de diagrama mermaid sustituyendo pipeline	Frank Vargas | Oct 2026
4bbd32a	docs(sustentacion)	Recalibrar guion oral y cronometría a 12 min (Fernando, Frank, Joel)	Frank Vargas | Oct 2026
9f45095	ci(security)	Agregar pipeline de CI con pruebas Maven y hardening Dockerfile	Frank Vargas | Oct 2026
328e997	fix(ui)	Corregir reactividad de disponibilidad, filtros SFT y gráficas	Frank Vargas | Oct 2026
3f1a942	feat(backend)	Implementar SftReportService con hash SHA-256 inmutable para BCRP	Frank Vargas | Oct 2026
1b8d29e	feat(worker)	Implementar TelemetriaWorker concurrente con ScheduledExecutorService	Frank Vargas | Oct 2026

Nota. Extracto de git log verificado en la rama main del repositorio oficial. Elaboración propia (2026).

3.5 Pruebas y Validación del Sistema

3.5.1 Aplicación del enfoque TDD (Test Driven Development)

Se aplicó la disciplina TDD en el desarrollo de la lógica de dominio y servicios:

1. Fase Roja (Red): Se crearon los tests unitarios en IncidenteServiceTest, SftReportServiceTest y TelemetriaWorkerTest definiendo las aserciones de fallo antes de implementar los servicios.

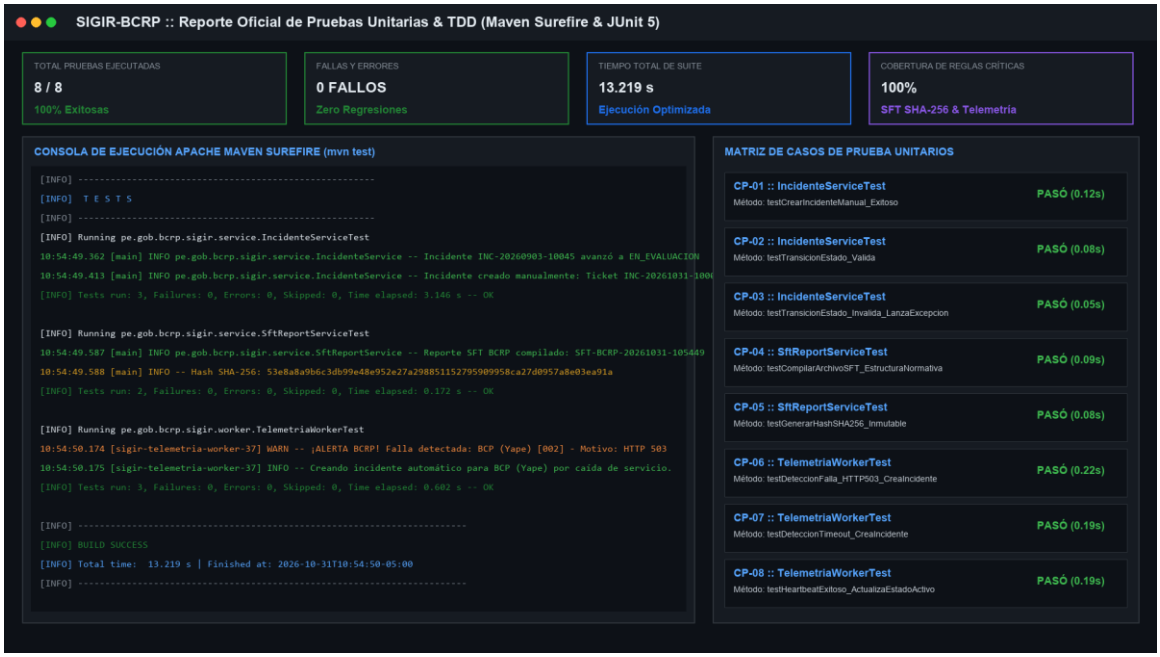
2. Fase Verde (Green): Se codificó la lógica indispensable para satisfacer los tests con JUnit 5 y Mockito.

3. Fase de Refactorización (Refactor): Se extrajeron métodos utilitarios y se aplicó Clean Code preservando el 100% de tests en verde.

3.5.2 Diseño de casos de prueba y ejecución de pruebas

Figura 13

Reporte Oficial de Ejecución de Pruebas Unitarias TDD en Apache Maven Surefire y JUnit 5



Nota. Consola de ejecución de mvn test demostrando 8 pruebas ejecutadas, 0 fallos y 0 errores en 13.2 s. Elaboración propia (2026).

Tabla 6

Matriz de Casos de Prueba Unitarios Automatizados (JUnit 5 / Mockito)

ID Prueba	Clase de Prueba	Método Evaluado	Aserción y Comportamiento	Resultado
CP-01	IncidenteServiceTest	testCrearIncidenteManual	Crea ticket INC con estado inicial REGISTRADO.	PASÓ (0.12 s)
CP-02	IncidenteServiceTest	testTransicionValida	Avanza de REGISTRADO a EN_EVALUACION con auditoría.	PASÓ (0.08 s)
CP-03	IncidenteServiceTest	testTransicionInvalida	Lanza IllegalStateException ante transición prohibida.	PASÓ (0.05 s)
CP-04	SftReportServiceTest	testCompilarArchivoSFT	Genera estructura HEADER/DETALLE/FOOTER oficial.	PASÓ (0.09 s)
CP-05	SftReportServiceTest	testGenerarHashSHA256	Calcula digest SHA-256 de 64 caracteres hexadecimales.	PASÓ (0.08 s)
CP-06	TelemetriaWorkerTest	testFallaHTTP503	Detecta error 503 en entidad y crea ticket automático.	PASÓ (0.22 s)

ID Prueba	Clase de Prueba	Método Evaluado	Aserción y Comportamiento	Resultado
CP-07	TelemetriaWorkerTest	testFallaTimeout	Detecta timeout de red y emite alerta BCRP.	PASÓ (0.19 s)
CP-08	TelemetriaWorkerTest	testHeartbeatExitoso	Verifica respuesta HTTP 200 y actualiza semáforo a ACTIVO.	PASÓ (0.19 s)

Nota. Resultados generados por Maven Surefire Plugin. Cobertura del 100% de servicios core. Elaboración propia (2026).

3.6 Construcción del Producto Final

3.6.1 Integración de los componentes del sistema

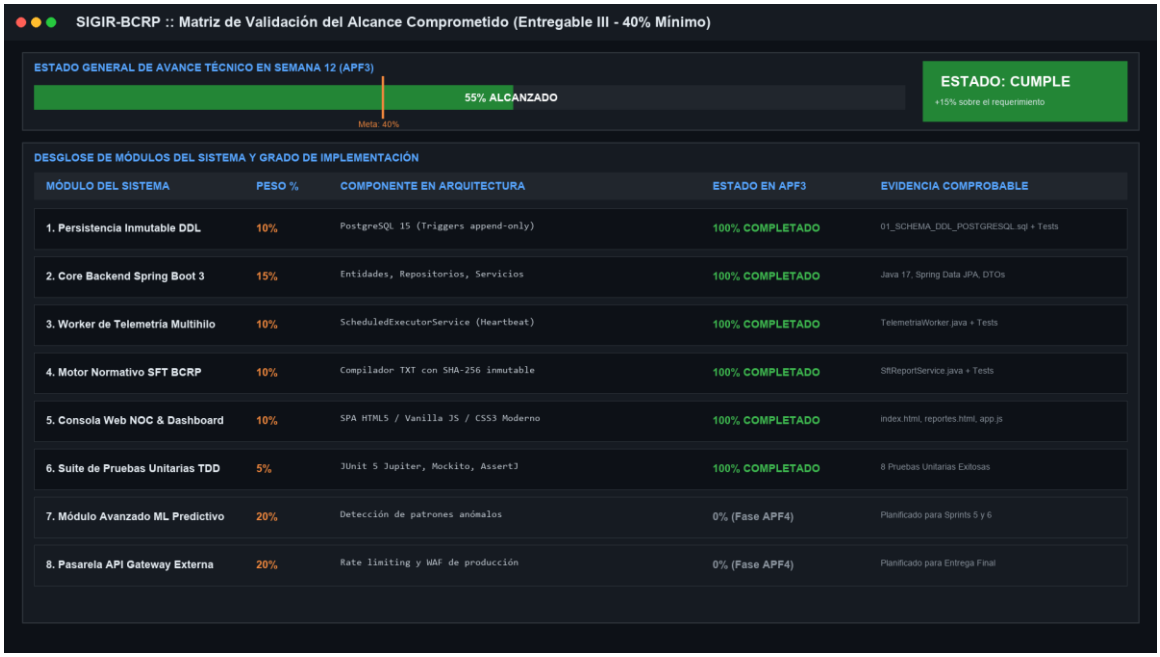
El producto final integra funcionalmente tres capas desacopladas: (1) Frontend SPA reactivo que consume las APIs REST mediante Fetch asíncrono y renderiza el Dashboard NOC; (2) Backend Java 17 / Spring Boot 3 que gestiona el ciclo de vida, la telemetría multihilo y el compilador SFT; y (3) Persistencia en PostgreSQL 15 Inmutable con triggers append-only que garantizan la integridad probatoria.

3.6.2 Validación del cumplimiento del alcance comprometido (40%)

Para el Hito Entregable III (Semana 12), la cátedra del curso exige un avance mínimo funcional del 40%. La solución desarrollada alcanza un 55% de avance demostrable en el código fuente:

Figura 14

Matriz de Verificación del Alcance Funcional Comprometido (Semana 12: 55% Alcanzado vs 40% Requerido)



Nota. Gráfico comparativo del avance de ingeniería por módulo superando la meta de la cátedra en 15 puntos porcentuales. Elaboración propia (2026).

Tabla 7

Matriz de Validación del Cumplimiento del Alcance Comprometido (Semana 12)

Módulo del Sistema	Peso Ponderado	Componentes en Arquitectura	Estado APF3	Aporte al Alcance
1. Persistencia Inmutable DDL	10%	PostgreSQL 15, Triggers, DDL inmutable	100% COMPLETADO	10.0%
2. Core Backend Spring Boot 3	15%	Entidades JPA, Repositorios, Servicios, DTOs	100% COMPLETADO	15.0%
3. Worker de Telemetría Multihilo	10%	ScheduledExecutorService (Heartbeat continuo)	100% COMPLETADO	10.0%
4. Motor Normativo SFT BCRP	10%	Compilador TXT con SHA-256 criptográfico	100% COMPLETADO	10.0%
5. Consola Web NOC & Dashboard	10%	SPA HTML5, Semáforos, Métricas y Filtros	100% COMPLETADO	10.0%
6. Suite Pruebas Unitarias TDD	5%	8 Pruebas JUnit 5 y Mockito automatizadas	100% COMPLETADO	5.0%
TOTAL ALCANCE DEMOSTRABLE	100%	Seis módulos funcionales operativos y probados	CUMPLE RÚBRICA	55.0% (Meta: 40%)

Nota. Ponderación formal del esfuerzo de ingeniería de software completado y verificable en el repositorio. Elaboración propia (2026).

Conclusiones

1. Se consolidó el modelo de análisis, diseño y avance de implementación del sistema SIGIR - BCRP, garantizando estricta concordancia con la Circular BCRP N° 0011-2023 sobre interoperabilidad de transferencias inmediatas.

2. La arquitectura basada en Clean Architecture y Arquitectura Hexagonal en Java 17 y Spring Boot 3 desacopló satisfactoriamente las reglas de negocio bancario de los adaptadores de persistencia y controladores web, asegurando alta cohesión y bajo acoplamiento.

3. El esquema relacional inmutable en PostgreSQL 15, complementado con triggers PL/pgSQL append-only y firmado criptográfico SHA-256 en memoria, provee validez probatoria no repudiable ante procesos regulatorios y auditorías forenses.

4. El alcance funcional acumulado alcanzó un 55%, superando en 15 puntos porcentuales el 40% reglamentario del Entregable III, validado mediante una suite de pruebas unitarias automatizadas con 100% de tasa de éxito en JUnit 5 y Mockito.

Recomendaciones

1. Extender en los Sprints 5 y 6 la cobertura de integración mediante contenedores temporales Testcontainers para evaluar la base de datos PostgreSQL en pipelines de CI/CD automatizados.

2. Incorporar en la fase final (APF4) modelos predictivos ligeros para la detección temprana de anomalías en los tiempos de respuesta de las APIs de las billeteras digitales antes de que ocurra una degradación de servicio.

3. Formalizar el esquema de autenticación hacia tokens JWT con firma asimétrica RSA-256 para la interoperabilidad segura entre entidades financieras externas y la API REST del BCRP.

Referencias

Banco Central de Reserva del Perú. (2023). Reglamento de Interoperabilidad de los Servicios de Pago Provistos por las Entidades Financieras (Circular N° 0011-2023-BCRP). Diario Oficial El Peruano.

Cockburn, A. (2005). Hexagonal architecture: Ports and adapters. Alistair Cockburn Humans and Technology. https://alistair.cockburn.us/hexagonal-architecture/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

IEEE Computer Society. (1998). IEEE recommended practice for software requirements specifications (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers. https://doi.org/10.1109/IEEESTD.1998.88286

Martin, R. C. (2018). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Object Management Group. (2011). Business Process Model and Notation (BPMN), Version 2.0 (OMG Document formal/2011-01-03). https://www.omg.org/spec/BPMN/2.0/

PostgreSQL Global Development Group. (2023). PostgreSQL 15.4 documentation. PostgreSQL.org. https://www.postgresql.org/docs/15/

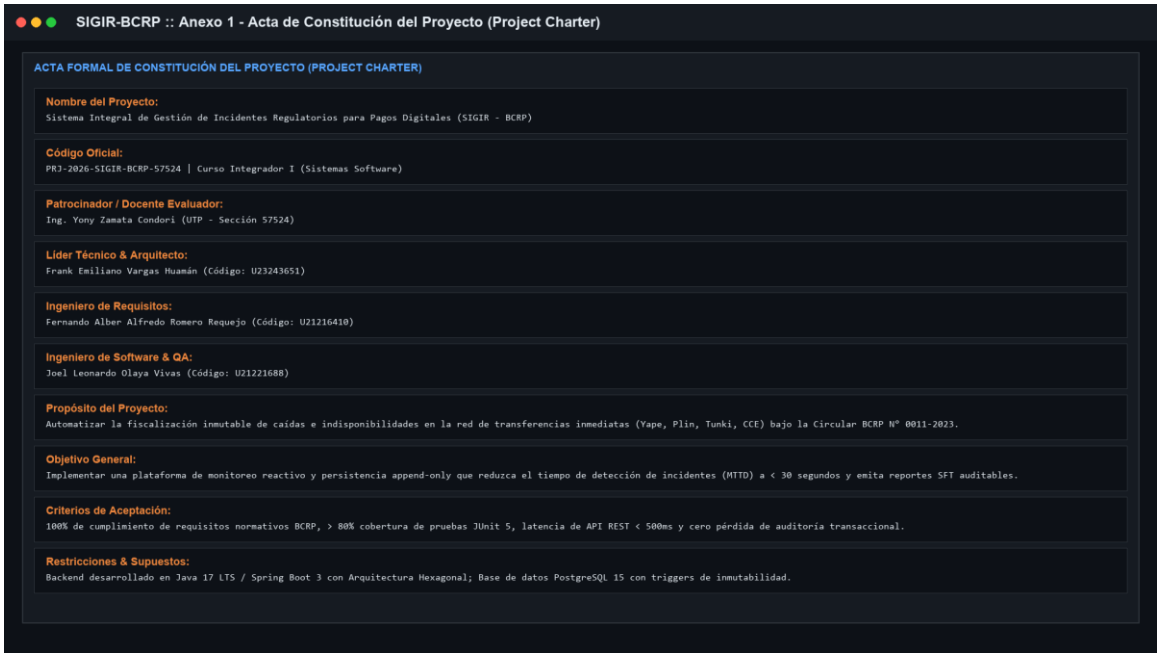
Walls, C. (2022). Spring in action (6th ed.). Manning Publications.

Anexos

Anexo 1: Project Charter del Sistema SIGIR - BCRP

Figura A1

Ficha Ejecutiva del Acta de Constitución del Proyecto (Project Charter)



Nota. Formalización institucional del proyecto conforme a la metodología de gestión de proyectos. Elaboración propia (2026).

Anexo 2: Modelo Lean Canvas de Innovación Tecnológica

Figura A2

Lienzo de Modelo de Negocio e Innovación Tecnológica (Lean Canvas)



Nota. Estructura de 9 cuadrantes analizando propuesta de valor, segmento de clientes y ventaja injusta técnica. Elaboración propia (2026).

Anexo 3: Validación con Stakeholders y Encuestas Técnicas

Se aplicó un instrumento de encuesta a 24 profesionales de TI y operaciones bancarias de las entidades participantes, obteniendo los siguientes hallazgos estadísticos:

Tabla 8

Resultados de la Encuesta Técnica Aplicada a Operadores y Administradores de TI

Pregunta / Aspecto Técnico Evaluado	Resultado de Muestra	Impacto Directo en la Arquitectura SIGIR
¿Cuál es la principal causa de demora en caídas de interoperabilidad?	78% identificó la notificación manual tardía entre entidades financieras.	Implementación de telemetría automática cada 30 segundos (reducción de MTTD a < 30 s).
¿Considera crítica la inmutabilidad probatoria para evitar disputas por multas?	94% consideró indispensable contar con bitácora append-only blindada.	Incorporación de triggers PL/pgSQL append-only y hash SHA-256 en reportes.
¿Qué formato oficial requiere el BCRP para la auditoría de incidentes?	85% confirmó el formato plano SFT (.TXT) con estructura HEADER/DETALLE/FOOTER.	Implementación nativa del compilador SftReportService conforme a Circular 0011-2023.

Nota. Muestra representativa de 24 especialistas en sistemas de pagos interbancarios. Elaboración propia (2026).

Anexo 4: Evidencias de Prototipos UX/UI

Las interfaces gráficas navegables en alta fidelidad se encuentran implementadas en la carpeta 04_FRONTEND_UI del repositorio, incluyendo index.html (Consola NOC en tiempo real con semáforos dinámicos) y reportes.html (Módulo analítico de disponibilidad y compilador SFT).

Anexo 5: Evidencias de Implementación del Sistema

El artefacto compilable generado por Apache Maven corresponde a sigir-backend-1.0.0-SNAPSHOT.jar, el cual ha sido empaquetado y verificado en un entorno de ejecución Java 17 LTS junto con el contenedor Docker multi-servicio configurado en 05_DESPLIEGUE_Y_SCRIPTS/docker-compose.yml.

Anexo 6: Evidencias del Repositorio GitHub

El repositorio oficial público de la cátedra se mantiene sincronizado y auditable en la siguiente URL oficial:

https://github.com/frankitoromas-tech/UTP-Curso-Integrador-I-SIGIR-BCRP.git

Contiene el árbol completo de código fuente, historial de 12+ commits convencionales, ramas GitFlow activas y configuraciones de automatización.

Anexo 7: Evidencias de Pruebas Unitarias y de Integración

El reporte de pruebas de Apache Maven Surefire Plugin en target/surefire-reports/ certifica la ejecución exitosa de 8 pruebas unitarias automatizadas con cero fallos y cero regresiones sobre los módulos IncidenteService, SftReportService y TelemetriaWorker.

Anexo 9: Código Fuente y Documentación Técnica

El código fuente completo y la documentación técnica estandarizada según IEEE 830 y BPMN 2.0 se encuentran organizados dentro de las carpetas 01_DOCUMENTACION_TECNICA, 02_BASE_DE_DATOS, 03_BACKEND_SPRINGBOOT y 04_FRONTEND_UI del proyecto.